Java Backend Architecture

Interview Series

Chapter 18.4

SSL/TLS Termination

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.4 – SSL/TLS Termination

Overview

Nginx is the standard TLS termination point for Java microservices, handling certificate management, cipher negotiation, and session resumption before forwarding plaintext to backend services. Correct TLS configuration is critical for security and performance. Understanding HSTS, OCSP stapling, session resumption, and mutual TLS is expected at senior level.

Key Definitions

Term	Definition
ssl_certificate	Path to PEM file containing server cert + full chain (leaf + intermediates). Not root CA.
ssl_certificate_key	Path to private key. Should be 0400 permissions, owned by nginx user.
ssl_protocols	Protocol versions allowed. Production: TLSv1.2 TLSv1.3 (drop TLSv1.0, TLSv1.1).
ssl_ciphers	Ordered cipher suite list. Use Mozilla SSL Config Generator modern profile.
ssl_session_cache	Shared memory for TLS session IDs across workers. shared:SSL:10m caches ~40k sessions.
ssl_session_tickets	Server encrypts session state into ticket sent to client. Stateless resumption.
OCSP Stapling	Server fetches and caches OCSP response; bundles with handshake. Avoids client OCSP lookup.
HSTS	HTTP Strict Transport Security: browser enforces HTTPS for max-age duration.
SNI	Server Name Indication: TLS extension for multiple certificates on one IP.
mTLS	Mutual TLS: both parties present certificates. Used for service-to-service auth.

Diagram 1: TLS Handshake with Session Resumption

```
Full Handshake (first connection):
Client                              Nginx
  |-- ClientHello (TLS 1.3) ------->|
  |<-- ServerHello + Certificate ---|
  |<-- CertificateVerify -----------|
  |-- Finished -------------------->| (session ticket issued)
  |<-- HTTP response ---------------|

Resumed Handshake (TLS 1.3 -- 1-RTT or 0-RTT with early data):
Client                              Nginx
  |-- ClientHello + session_ticket ->|
  |<-- Finished --------------------|
  |-- HTTP request ---------------->|
```

Diagram 2: Certificate Chain

```
ssl_certificate fullchain.pem:
+---------------------------+
|  Leaf Certificate         |  <- api.example.com
|  (your domain cert)       |
+---------------------------+
|  Intermediate CA          |  <- Let's Encrypt R3 / DigiCert CA
+---------------------------+
|  (Root CA omitted --      |  <- already in browser trust stores
|   not needed in chain)    |
+---------------------------+
ssl_certificate_key privkey.pem  (private key, separate file)

Verify: openssl verify -CAfile chain.pem cert.pem
Check expiry: openssl x509 -enddate -noout -in cert.pem
```

Code Examples

Production TLS Config (Mozilla Modern Profile):

```
server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate     /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/privkey.pem;

    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
                        ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
                        ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;

    ssl_session_cache   shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets on;

    ssl_stapling        on;
    ssl_stapling_verify on;
    ssl_trusted_certificate /etc/nginx/ssl/chain.pem;
    resolver 8.8.8.8 valid=300s;

    add_header Strict-Transport-Security
      "max-age=63072000; includeSubDomains; preload" always;

    location / { proxy_pass http://java_backend; }
}

server {
    listen 80;
    server_name api.example.com;
    return 301 https://$host$request_uri;
}
```

Mutual TLS (mTLS) for Service-to-Service:

```
server {
    listen 443 ssl http2;
    ssl_certificate     /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;
    ssl_client_certificate /etc/nginx/ssl/ca.crt;
    ssl_verify_client   on;

    location /internal-api/ {
        proxy_set_header X-Client-Cert $ssl_client_escaped_cert;
        proxy_set_header X-Client-DN   $ssl_client_s_dn;
        proxy_pass http://java_backend/;
    }
}
```

cert-manager + Let's Encrypt (Kubernetes):

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: letsencrypt-prod }
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef: { name: letsencrypt-prod }
    solvers: [{ http01: { ingress: { class: nginx } } }]
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  tls: [{ hosts: [api.example.com], secretName: api-tls }]
```

Interview Q&A – SSL/TLS Termination

Q1. What is the difference between ssl_session_cache and ssl_session_tickets?

ssl_session_cache stores session IDs server-side in shared memory (all workers share). Client presents session ID on reconnect; server looks up cached parameters -- avoids full TLS handshake. ssl_session_tickets: server encrypts full session state into a ticket sent to client; stateless (no server storage). TLS 1.3 uses tickets exclusively. Both can coexist; together they provide maximum session resumption coverage.

Q2. Why set ssl_prefer_server_ciphers off for TLS 1.3?

TLS 1.3 has only 5 cipher suites, all secure -- client preference is fine. ssl_prefer_server_ciphers matters for TLS 1.2 where clients might choose weaker ciphers. For TLS 1.2: on enforces server's strong cipher order. For TLS 1.3: off lets client select its hardware-optimised cipher (AES-GCM on AES-NI, ChaCha20 on mobile without AES-NI). Mozilla modern profile sets off.

Q3. What is OCSP stapling and why does it improve performance?

Without stapling: client queries CA's OCSP responder during handshake -- adds 50-200ms latency and leaks client IPs to CA. With ssl_stapling on: Nginx fetches and caches OCSP response, staples it to the TLS handshake. Client gets revocation status without external lookup. ssl_stapling_verify on validates OCSP response signature. Requires ssl_trusted_certificate with the full chain (excluding leaf).

Q4. Explain HSTS and the preload list.

Strict-Transport-Security: max-age=63072000 tells browsers to use only HTTPS for 2 years. includeSubDomains applies to all subdomains. preload submits the domain to Chrome's hardcoded HSTS list -- browsers enforce HTTPS before first connection (prevents HSTS stripping on first visit). WARNING: preload is very hard to undo. Ensure ALL subdomains support HTTPS before enabling.

Q5. How does SNI enable multiple SSL certificates on one IP?

SNI: TLS ClientHello extension includes the hostname. Nginx reads SNI before handshake completes (before decryption) to select the correct ssl_certificate per virtual host. Multiple server blocks with different server_names and ssl_certificates on the same listen 443 ssl port -- Nginx selects the right cert per request. Critical for multi-tenant Java SaaS deployments with per-customer certificates.

Q6. Why must ssl_certificate include the full chain?

Leaf-only certificate: clients cannot verify it without the intermediate CAs. Some clients (curl, some Java HTTP clients) fail with 'certificate chain incomplete'. fullchain.pem = leaf + intermediates (not root -- that's in trust stores). Verify chain completeness: openssl s_client -connect host:443 -showcerts. Missing intermediates cause intermittent failures depending on client OS certificate cache state.

Q7. How do you handle certificate rotation with zero downtime?

Replace ssl_certificate/ssl_certificate_key files and run nginx -s reload. New workers use new cert; old workers drain with old cert -- no connection drops. Let's Encrypt + certbot: post-hook calls nginx -s reload after renewal. Kubernetes: cert-manager updates the TLS Secret; Nginx Ingress Controller detects Secret change and reloads. Automate with monitoring on certificate expiry (prometheus ssl_certificate_expiry metric).

Q8. What ssl variables does Nginx expose for Java backends?

$ssl_protocol (TLSv1.3), $ssl_cipher (current cipher suite), $ssl_client_s_dn (client cert DN for mTLS), $ssl_client_verify (SUCCESS/FAILED/NONE), $ssl_client_escaped_cert (URL-encoded PEM for forwarding), $ssl_session_reused (r=reused). Log $ssl_protocol to monitor TLS 1.0/1.1 usage before deprecating. Pass client cert info to Java backend via proxy_set_header for audit logging.

Q9. Why must TLSv1.0 and TLSv1.1 be disabled?

TLS 1.0/1.1 deprecated by IETF (RFC 8996). Vulnerabilities: POODLE (CBC padding oracle in TLS 1.0), BEAST. PCI DSS and NIST SP 800-52r2 prohibit TLS 1.0 for cardholder data. Java JDK 11+ disables TLS 1.0/1.1 by default in client. ssl_protocols TLSv1.2 TLSv1.3 is the correct production setting. Test: openssl s_client -connect host:443 -tls1_1 should return handshake failure.

Q10. How do you implement TLS passthrough at the stream layer?

In stream block (TCP proxy): server { listen 443; ssl_preread on; proxy_pass $backend_host; } map $ssl_preread_server_name $backend_host { api.example.com api-backend:443; app.example.com app-backend:443; } ssl_preread parses ClientHello SNI without terminating TLS, routing based on hostname. Nginx never sees decrypted data -- Java backend handles TLS. Required

when private keys must not be on Nginx.

Q11. What is ssl_dhparam and when is it needed?

Diffie-Hellman parameters for DHE key exchange (TLS 1.2 DHE cipher suites). OpenSSL defaults to 1024-bit DH (too weak -- breaks on modern browsers). Generate: openssl dhparam -out dhparam.pem 2048. TLS 1.3 uses ECDHE exclusively, so ssl_dhparam only matters for TLS 1.2. Modern configs prefer ECDHE suites -- DHE is rarely needed. Include for defence-in-depth if TLS 1.2 DHE suites are in ssl_ciphers.

Q12. How do you test SSL/TLS configuration?

External: SSL Labs (ssllabs.com/ssltest) gives A-F grade with detailed vulnerability analysis. testssl.sh: open-source CLI checks protocols, ciphers, vulnerabilities (BEAST, POODLE, HEARTBLEED). openssl s_client -connect host:443 -showcerts shows full chain. curl --tls-max 1.1 tests TLS version restriction. nmap --script ssl-enum-ciphers. Automate in CI to catch certificate expiry and config regressions.